

How to download data from an Elasticsearch Export index

Aman Sharma • Last Modified: September 5th, 2023

Data export and analytics are on the rise for all brands intending to analyze structured and unstructured data to optimize and fine-tune their business strategies. Elasticsearch is one of the many platforms that are gaining popularity in the analytics domain.

This article deals with methods to export data from Elasticsearch and uses in tandem with platforms, along with detailed code snippets to implement the same. It also throws some light on the challenges of manual scripting for data export along with possible solutions to overcome them.

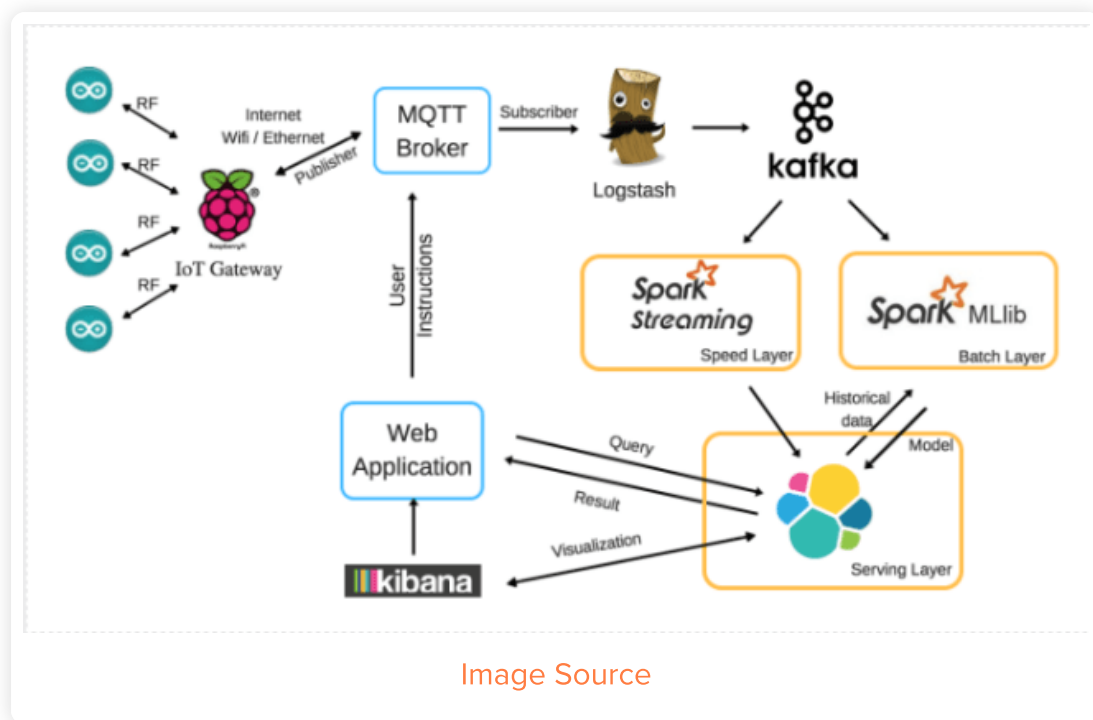
List of Contents

- [What is Elasticsearch?](#)
- [Why Export Elasticsearch Data?](#)
- [Elasticsearch Export Methods](#)
 - [Elasticsearch Export: Using Logstash-Input-Elasticsearch Plugin](#)
 - [Elasticsearch Export: Using Elasticsearch Dump](#)
 - [Elasticsearch Export: Using Python Pandas](#)
- [Limitations of Manual Export](#)
- [Conclusion](#)

What is Elasticsearch?

Elasticsearch is an open-source search and analytics engine that has a robust REST API, a distributed nature, and ample speed and scalability for use, across multiple platforms. It allows the analytics of textual, numerical, and even geospatial data that can be employed for any intended use.

It also facilitates data ingestion, storage, analysis, enrichment, and visualization in the most comprehensive forms. With bulk datasets, the issue of data export, which is reliable and fast becomes a valid concern.



Why Export Elasticsearch Data?

While data in Elasticsearch can be manipulated and enhanced to derive beneficial insights, these metrics often need to be transmitted to other platforms that a company is using for storage or other data manipulation purposes.

Thus, data integrations and export from Elasticsearch is a phenomenon that is widely employed by companies to get the data in their desired storage or warehouse locations.



Image Source

Hevo Data: Export your Elasticsearch Data Conveniently

Hevo is a No-Code Data Pipeline. It supports pre-built data integrations from **100+ data sources**, including Elasticsearch.

GET STARTED WITH HEVO FOR FREE

Hevo offers a fully managed solution for your data migration process. It will automate your data flow in minutes without writing any line of code. Its fault-tolerant architecture makes sure that your data is secure and consistent. Hevo provides you with a truly efficient and fully automated solution to manage data in real-time and always have analysis-ready data at your data warehouse.

Let's look at some salient features of Hevo:

- **Fully Managed:** It requires no management and maintenance as Hevo is a fully automated platform.
- **Data Transformation:** It provides a simple interface to perfect, modify, and enrich the data you want to transfer.
- **Real-Time:** Hevo offers real-time data migration. So, your data is always ready for analysis.

- **Schema Management:** Hevo can automatically detect the schema of the incoming data and map it to the destination schema.
- **Live Monitoring:** Advanced monitoring gives you a one-stop view to watch all the activities that occur within pipelines.
- **Live Support:** Hevo team is available round the clock to extend exceptional support to its customers through chat, email, and support calls.

SIGN UP HERE FOR A 14-DAY FREE TRIAL!



Download the Ultimate Guide on Database Replication

Download the Ultimate Guide on Database Replication

Learn the 3 ways to replicate databases & which one you should prefer.

GET GUIDE FOR FREE →

Elasticsearch Export Methods

There are several ways in which you can export data from Elasticsearch, depending upon the type of file you want to export. Whether you want to export your data as a CSV file, a table in HTML, or a JSON file, it depends on the use case of your company.

Here are three popular methods, you use to export files from Elasticsearch to any desired warehouse or platform of your choice:

- [Elasticsearch Export: Using Logstash-Input-Elasticsearch Plugin](#)
- [Elasticsearch Export: Using Elasticsearch Dump](#)
- [Elasticsearch Export: Using Python Pandas](#)

1. Export Elasticsearch Data: Using Logstash-Input-Elasticsearch Plugin

There are several plug-ins that you can use for rapid data export from Elasticsearch. If you are looking for data export as CSV files, this method can be useful.

Run the following script to install the Logstash-Input-Elasticsearch Plugin:

```
# cd /opt/logstash/
# bin/plugin install logstash-input-elasticsearch
The use of bin/plugin is deprecated and will be removed
in a feature release. Please use bin/logstash-plugin.
Validating logstash-input-elasticsearch
Installing logstash-input-elasticsearch
Installation successful
```

After this you will also require to install the Logstash-Output-CSV plugin to receive the outputs in the desired format:

```
# cd /opt/logstash
# bin/logstash-plugin install logstash-output-csv
Validating logstash-output-csv
Installing logstash-output-csv
Installation successful
```

Now, you can write your queries in the input section to return the JSON values as a CSV output file. The output plugin will pick out specific fields that are configured under logstash to save CSV files directly as a result of the query.

This configuration can be seen in the following script:

```
input {
  elasticsearch {
    hosts => "localhost:9200"
    index => "index-we-are-reading-from1"
    query => '
    {"query": {
      ..
      #Insert your Elasticsearch query here
    }
  }
}
```

```
    }
  }
  }
}
output {
  csv {
    # This is the field that you would like to output
    # in CSV format.
    # The field needs to be one of the fields shown in the output
    # when you run your
    # Elasticsearch query
    fields => ["field1", "field2", "field3",
              "field4", "field5"]
    # This is where we store output.
    We can use several files to store our output
    # by using a timestamp to determine
    # the filename where to store output.
    path => "/tmp/csv-export.csv"
  }
}
```

2. Elasticsearch Export: Using Elasticsearch Dump

Another way to effectively initiate an export in the desired format is by using the Elasticsearch dump. Elasticdump alters the format of dump files into the desired format which can then be scanned and recorded in the specified location.

.....

```
npm install elasticdump
./bin/elasticdump
```



OR

```
npm install elasticdump -g
elasticdump
```



After installation, you can specify input and output for the process, each of which can either be URLs or files. Data export using elasticsearchdump is shown below:

```
# Export ES data to S3 (using s3urls)
elasticsearchdump
  --s3AccessKeyId "${access_key_id}"
  --s3SecretAccessKey "${access_key_secret}"
  --input=http://production.es.com:9200/my_index
  --output "s3://${bucket_name}/${file_name}.json"

# Export ES data to MINIO (s3 compatible) (using s3urls)
elasticsearchdump
  --s3AccessKeyId "${access_key_id}"
  --s3SecretAccessKey "${access_key_secret}"
  --input=http://production.es.com:9200/my_index
  --output "s3://${bucket_name}/${file_name}.json"
  --s3ForcePathStyle true
  --s3Endpoint https://production.minio.co
```

You can also use MultiElasticDump to run a similar script for multiple indices at once:

```
# backup ES indices & all their type to the es_backup folder
multielasticdump
  --direction=dump
  --match='^.*$'
  --input=http://production.es.com:9200
  --output=/tmp/es_backup

# Only backup ES indices ending with a prefix of `-index` (match
# regex).
# Only the indices data will be backed up. All other types are
# ignored.
# NB: analyzer & alias types are ignored by default
multielasticdump
  --direction=dump
  --match='^.*-index$'
  --input=http://production.es.com:9200
```

```
--ignoreType='mapping,settings,template'  
--output=/tmp/es_backup
```

3. Elasticsearch Export: Using Python Pandas

Python Pandas can be used to export documents in various formats. You can use Elasticsearch Pandas to export files in HTML, CSV or JSON formats.

Install Python 3's PIP using the following command:

```
sudo apt install python3-pip  
sudo yum install python36  
sudo yum install python36-devel  
sudo yum install python36-setuptools  
sudo easy_install-3.6 pip
```



The complete script for exporting and initiating a data export in any of these forms using Python's Pandas is as follows:

```
#!/usr/bin/env python3  
#-*- coding: utf-8 -*-  
  
import sys, time, io  
start_time = time.time()  
  
if sys.version[0] != "3":  
    print ("nThis script requires Python 3")  
    print ("Please run the script using the 'python3' command.n")  
    quit()  
try:  
    # import the Elasticsearch low-level client library  
    from elasticsearch import Elasticsearch  
    # import Pandas, JSON, and the NumPy library  
    import pandas, json  
    import numpy as np  
except ImportError as error:  
    print ("nImportError:", error)  
    print ("Please use 'pip3' to install the necessary packages.")  
    quit()
```




```
# create a client instance of the library
print ("ncreating client instance of Elasticsearch")
elastic_client = Elasticsearch()

"""
MAKE API CALL TO CLUSTER AND CONVERT
THE RESPONSE OBJECT TO A LIST OF
ELASTICSEARCH DOCUMENTS
"""

# total num of Elasticsearch documents to get with API call
total_docs = 20
print ("nmaking API call to Elasticsearch for", total_docs,
"documents.")
response = elastic_client.search(
    index='employees',
    body={},
    size=total_docs
)
# grab list of docs from nested dictionary response
print ("putting documents in a list")
elastic_docs = response["hits"]["hits"]

"""
GET ALL OF THE ELASTICSEARCH
INDEX'S FIELDS FROM _SOURCE
"""

# create an empty Pandas DataFrame object for docs
docs = pandas.DataFrame()
# iterate each Elasticsearch doc in list
print ("ncreating objects from Elasticsearch data.")
for num, doc in enumerate(elastic_docs):
    # get _source data dict from document
    source_data = doc["_source"]
    # get _id from document
    _id = doc["_id"]
    # create a Series object from doc dict object
    doc_data = pandas.Series(source_data, name = _id)
    # append the Series object to the DataFrame object
    docs = docs.append(doc_data)

"""
EXPORT THE ELASTICSEARCH DOCUMENTS PUT INTO
PANDAS OBJECTS
"""

print ("nexporting Pandas objects to different file types.")
```

```
# export the Elasticsearch documents as a JSON file
docs.to_json("objectrocket.json")
# have Pandas return a JSON string of the documents
json_export = docs.to_json() # return JSON data
print ("nJSON data:", json_export)
# export Elasticsearch documents to a CSV file
docs.to_csv("objectrocket.csv", ",") # CSV delimited by commas
# export Elasticsearch documents to CSV
csv_export = docs.to_csv(sep=",") # CSV delimited by commas
print ("nCSV data:", csv_export)
# create IO HTML string
import io
html_str = io.StringIO()
# export as HTML
docs.to_html(
    buf=html_str,
    classes='table table-striped'
)

# print out the HTML table
print (html_str.getvalue())
# save the Elasticsearch documents as an HTML table
docs.to_html("objectrocket.html")

print ("nntime elapsed:", time.time()-start_time)
```

Code Snippets from [Qbox](#), [GitHub](#) and [ObjectRocket](#)

Limitations of Manual Export

While the above-listed methods for a manual export are easy to implement and follow-through, there are known limitations to this manual export process. Firstly, in-depth technical knowledge is required to be able to install the required plug-ins and implement these scripts. Although many professionals might find this task achievable, non-tech sectors that desire to export their data might not readily be able to do so.

Secondly, any manual integration holds the possibility of errors and inconsistent or invalid code that can cause problems in the general process. If implemented incorrectly, there is also a possibility to lose data and corrupt important files. These issues can be resolved with the use of platforms that are not based on code implementation and can automate the export process.

Conclusion

In this blog, you learned about Elasticsearch export using three different methods. You can select your desired method to export Elasticsearch data for robust use across multiple platforms. Running scripts via selected plug-ins can help carry out this task, while other methods will automate the process for you. Choose your desired channel to reap the benefits of Elasticsearch in tandem with your data warehouse. But, if you are looking for an automated solution to export data from Elasticsearch to a data warehouse, then try Hevo.

Hevo is a No-Code Data Pipeline. It supports pre-built integrations from **100+ data sources** at a reasonable **price**. With Hevo, you can migrate your Elasticsearch data to your data warehouse in minutes.

VISIT OUR WEBSITE TO EXPLORE HEVO

SIGN UP for a 14 day free trial.

Share your experience of exporting Elasticsearch data in the comment section below.

No-code Data Pipeline for your Data Warehouse

TRY FOR FREE

Elasticsearch

Continue Reading